

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962\agent-ae85f5aa4d361f8f5.jsonl`  
- SHA-256: `ffef37ac7e0254f1e6add2f519b62214dc730ad4b7bfe19729464a33f02d0ac9`  
- Source modified: `2026-06-07T08:15:15+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_49729ae7-962`  
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T08:13:58.709Z)

You are a USER of a sports-video tool who has read ONLY its README/FAQ ? you have NEVER seen the source code. A processing run looks wrong and the tool wrote a per-video report. Read EXACTLY these two files and NOTHING else (do not open source code or any other file):

1. The README/FAQ: C:\Users\avidu\Projects\badminton-highlight-indexer\README.md
2. The tool's technical report for the suspicious run:
C:\Users\avidu\AppData\Local\Temp\obseval\reports\indexer_motion_synthetic.report.md

Using ONLY those two, tell me: what went wrong, exactly what you'd do to fix it, and which FAQ section (by heading text or Q-number) you used. Be honest about whether you could truly self-debug this from the report alone, and list anything in the report that was confusing, ambiguous, or under-explained.

2. assistant (2026-06-07T08:14:00.190Z)

I'll read the two files you specified to diagnose the issue.

3. user (2026-06-07T08:14:00.196Z)

```
1      # Badminton Highlight & Rally Indexer  
2  
3      A local, AI-powered web application and CLI utility designed to process long-form  
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,  
integrity, and relevance checks), index all rallies into a relational SQLite database, and  
dynamically compile custom highlight reels using high-speed FFmpeg stitching.  
4  
5      The pipeline is built around three pluggable registries ? validators,  
segmenters, and AI providers ? plus a sport profile registry so the same engine can  
target other racket/court sports in the future. Badminton is the default profile.  
6  
7      > Where this is heading: the project is evolving from an AI-API-dependent prototype  
into a self-improving tool that learns rally detection offline from a commercially-licensed,  
annotated corpus. See [docs/ROADMAP.md](docs/ROADMAP.md) for the four-phase plan and  
[docs/DECISIONS.md](docs/DECISIONS.md) (ADR-002?004) for the rationale.  
8  
9      ---  
10  
11     ## ? Key Features  
12  
13     * Pluggable Video Validation Framework: Easily extendable base-class architecture  
that automatically runs checks on:  
14         * Static Format & Resolution: Verifies container is MP4, resolution >= 720p, and  
framerate >= 30 FPS.  
15         * PTS Timeline Continuity: Scans keyframe timestamps for timeline gaps, jumps,  
or editing splices to detect tampering or splices.
```

```

16      *   *Scene Cut Density Check*: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17      *   *Blurriness Check*: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18      *   *Relevance (Is it Badminton?): Uses high-performance HSV color court matching
to identify court layouts.
19      *   **Three Pluggable Rally Segmenters** (`backend/indexer/`):
20      *   `yolo_hybrid` ** (default) ? two-stage pipeline. Stage 1 runs **torchvision
person detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-
licensed) to cheaply filter the video down to a small set of high-motion candidate windows.
Stage 2 hands each candidate (with a small padding) to the AI provider for high-precision
semantic boundaries and shot counts. Massively cheaper than running the LLM on the full video.
*(The registry key remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced
with permissive components ? see [docs/MONETIZATION_AUDIT.md](docs/MONETIZATION_AUDIT.md) and
ADR-005.)*
21      *   `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22      *   `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23      *   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24      *   **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25      *   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26      *   **Query System**: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27      *   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28      *   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering.
29      *   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics,
validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31      ---
32
33      ## ?? Setup & Diagnostics
34
35      ### 1. External System Requirements
36      The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
37
38      *   **On Windows (PowerShell as Admin)**:
39      ```powershell
40      winget install Gyan.FFmpeg
41      ```
42      *Restart your terminal afterwards to register the path changes.*
43
44      ### 2. Auto-Installer & Diagnostics Checker
45      Run the diagnostics checker script in the root directory to verify your dependencies,
initialize default configs, and install PIP packages:
46      ```bash
47      python setup.py
48      ```
49      This script will:
50      *   Validate `ffmpeg` and `ffprobe` are present in your PATH.
51      *   Initialize a default `config.json` (if one does not exist) with thresholds for blur,
court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.

```

```

52     * Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching
PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
53     * Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`,
`numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`,
`supervision`.
54
55     > The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster
R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
`fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
`retinanet_resnet50_fpn`.)
56
57     ---
58
59     ## ? Running the Application
60
61     ### 1. Launching the Local Web App Dashboard
62     To start the FastAPI server and open the web dashboard:
63     ```bash
64     python -m backend.main --server --port 8000
65     ```
66     Open your browser and navigate to **[http://localhost:8000](http://localhost:8000)**.
67     *Simply input your local raw video file path in the sidebar to process, validate, index,
and compile highlights instantly.*
68
69     ### 2. Running in CLI Processing Mode
70     To process and index a video completely via terminal commands:
71     ```bash
72     python -m backend.main --video-path "C:/path/to/my_video.mp4"
73     ```
74     This outputs a validation summary and indexes all rallies directly into the local SQLite
database, using whichever segmenter is set as `default_segmenter` in `config.json`
(`yolo_hybrid` by default).
75
76     To explicitly pick a segmenter for one run:
77     ```bash
78     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
79     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
80     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
81     ```
82
83     ### 3. Rich CLI Debugging (`--debug-clip`)
84     To diagnose exactly why a video segment was registered as active/dead play, or inspect
detailed focus/continuity metrics at a particular time in seconds:
85     ```bash
86     python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
87     ```
88     *Outputs granular validation check logs and motion ratios within a +/- 3 second frame
window around the timestamp.*
89
90     ### 4. Extracting an Arbitrary Trimmed Segment (`--trim-start` / `--trim-end`)
91     For fast iteration on a tiny slice of a multi-GB recording, you can stream-copy a
segment to disk without re-encoding:
92     ```bash
93     python -m backend.main --video-path "C:/path/to/my_video.mp4" \
94         --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
95     ```
96     Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
path.
97
98     ### 5. End-to-End "Process + Stitch" Script
99     `run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a
hardcoded video path, reuses cached indexed segments if available (offering to re-index from
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching

```

```

settings against a fixed large recording without re-running expensive segmentation.
100
101     ---
102
103     ## ?? Threshold Tuning (`config.json`)
104     You can adjust the parameters of your validators, segmenters, AI provider, and stitcher
105     by editing `config.json` in the root folder. The default config emitted by `setup.py`:
106
107     ```json
108     {
109         "sport": "badminton",
110         "ai_provider": "gemini",
111         "ai_model": "gemini-2.5-flash",
112         "validation": {
113             "min_resolution_width": 1280,
114             "min_resolution_height": 720,
115             "min_fps": 29.9,
116             "min_blur_threshold": 20.0,
117             "max_scene_cuts_per_minute": 0.5,
118             "relevance": {
119                 "court_green_lower": [35, 40, 40],
120                 "court_green_upper": [85, 255, 255],
121                 "court_pixels_min_ratio": 0.05
122             }
123         },
124         "indexing": {
125             "default_segmenter": "yolo_hybrid",
126             "use_gpu": true,
127             "motion_threshold": 0.08,
128             "min_segment_duration": 3.0,
129             "dead_time_merge_gap": 2.0,
130             "chunk_duration": 90,
131             "chunk_overlap": 10,
132             "yolo_velocity_threshold": 0.15,
133             "yolo_frame_skip": 3,
134             "yolo_tracker": "bytetrack.yaml",
135             "yolo_parallel_chunks": false,
136             "yolo_chunk_workers": 2,
137             "min_action_window_duration": 2.0,
138             "ai_handoff_padding": 2.0,
139             "ai_max_workers": 5
140         },
141         "output": {
142             "default_highlights_name": "highlights.mp4",
143             "db_name": "sports_indexer.db"
144         },
145         "stitching": {
146             "begin_padding": 0.5,
147             "end_padding": 0.5,
148             "use_cache": false,
149             "min_shot_count": 1
150         }
151     }
152     ```
153
154     Notable knobs:
155     * `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for
156     noisy indoor-arena GoPro footage; see `TEST_RUN_SUMMARY.md` for why).
157     * `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
158     * `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.
159     * `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower
160     values catch slower play, higher values are stricter about what counts as "action".
161     * `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30
162     fps ? 10 samples/sec.
163     * `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO

```

```

phase.
160     * `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate
window before sending to the AI provider.
161     * `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
162     * `stitching.min_shot_count` ? drop play segments whose estimated shot count is below
this threshold when compiling highlights.
163
164     ---
165
166     ## ? Frequently Asked Questions (FAQ)
167
168     ### Q1: The server complains that `ffprobe` is not found.
169     Ensure FFmpeg is installed and added to your system environment variables. You can check
this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `setup.py`
outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.
170
171     ### Q2: Why does my video fail the Relevance check?
172     Our default relevance check scans for the typical indoor green badminton court floor
color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust
the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and
`court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to
bypass.
173
174     ### Q3: How do I add a new custom validator to the pipeline?
175     1. Create a new python file in `backend/validators/` (e.g. `my_check.py`).
176     2. Subclass `VideoValidator` and implement `validate(self, video_path, config,
context)`.
177     3. Register your class inside `backend/validators/__init__.py` using
`registry.register(MyCheckValidator())`.
178     It's that simple! The app will automatically run it, display it in the CLI diagnostics
summary, and render it in the frontend checklist.
179
180     ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
181     * Print Buffering: Standard Python buffers console output when stdout is
redirected or running in background shells on Windows. To get real-time unbuffered log
statements immediately, run commands with Python's unbuffered flag:
182     ```bash
183     python -u -m backend.main --video-path "C:/path/to/video.mp4"
184     ```
185     * Video Seeking Overhead: Seeking to random frames in high-bitrate long H.265/HEVC
videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The
`motion` segmenter reads sequentially and skips frame decodes using container-level
`cap.grab()`, which delivers a 100x speedup on multi-gigabyte raw files. `yolo_hybrid`
sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.
186
187     ### Q5: Why did the video fail with "No keyframes / packet timestamps could be
extracted"?
188     Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes.
Standard FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning
empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the
container demuxer level instead, which is incredibly robust and executes instantly without
decoding a single pixel.
189
190     ### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB
video file?
191     You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API
payload. This restricts the segmenter frame loop to process only the first $N$ sub-sampled
frames, enabling you to test motion filters and court relevance checks in seconds:
192     ```bash
193     python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-
frames 300
194     ```
195     *(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60
seconds of video).*
196

```

```

197     ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
198     Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use
Google Gemini:
199     1. Create a local `.env` file in the project root (this is already in `.gitignore` to
prevent secret leaks).
200     2. Set your Google AI Studio API key in the file:
201         ```.env
202         GEMINI_API_KEY=your_key_here
203         ```
204     3. Run with whichever segmenter you want:
205         ```bash
206         # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
207         python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
208
209         # Full-video Gemini analysis (more expensive, highest semantic precision per call)
210         python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
211
212         # Pure CV baseline ? no API key needed
213         python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
214         ```
215         *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"/"motion"`) under
`"indexing"` in `config.json`.
216
217     The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
`ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers
can be added under `backend/providers/`.
218
219     ### Q8: How does the `yolo_hybrid` pipeline actually save money?
220     Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs
in four phases:
221
222     1. Chunking ? splits the video into overlapping windows (`chunk_duration`,
`chunk_overlap`) so we can process incrementally.
223     2. Detection + tracking ? torchvision person detection (Faster R-CNN) + ByteTrack
runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity.
Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are
dropped. Adjacent active frames are merged into candidate windows, with
sub-`min_action_window_duration` windows discarded.
224     3. AI handoff ? only the surviving candidate windows (padded by `ai_handoff_padding`
seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and
shot-count estimation. These calls run in parallel up to `ai_max_workers`.
225     4. DB population ? confirmed segments are written to `play_segments` along with
`shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.
226
227     In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus
uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic
boundary detection.
228
229     ### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?
230     Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and
expensive. We built a native fast local trimming developer debugging feature:
231     1. Open your `config.json` file.
232     2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in
seconds):
233         ```json
234         "indexing": {
235             "default_segmenter": "gemini",
236             "debug_duration": 15.0,
237             ...
238         }
239         ```
240     3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-
encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds,
upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds

```

to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

241

242 ### Q10: Can I trust the per-segment metadata from the `motion` segmenter?

243 **No.** The zero-dependency `motion` segmenter detects *timing* via pixel-difference, but its per-segment *metadata* is heuristic/synthetic ? *intensity*, *avg_speed_kmh*, the serve/smash/foul/winner *events*, and *server*/*receiver* are generated values, *not measured*. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use *yolo_hybrid* (default) or *gemini*. If your observability report shows a *synthetic_motion_metadata* warning, this is why.

244

245 ### Q11: I re-ran a video and my previous results changed/disappeared. Why?

246 Videos are keyed by the *MD5* of the file's bytes. Re-processing the *same* file is treated as a fresh ingest: *add_video* is *INSERT OR REPLACE*, and the foreign-key cascade *deletes* the previous play/candidate segments before writing the new ones. So the earlier results are intentionally replaced, not merged. (A *reingest_replaced_prior* note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

247

248 ### Q12: What is the *report.json* / *report.md* / *summary.md* file next to my video?

249 Every processing run now writes a *per-video observability report* next to the video (or to *report.output_dir* if set in *config.json*):

250 - *report.json* ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.

251 - *report.md* ? the *technical* report: validation results, processing stages, and *findings/warnings*. Each warning names the problem and cites the FAQ entry to read. *Start* here when a run looks wrong.

252 - *summary.md* ? a *customer-friendly* summary: just the video metadata and highlights, no system internals.

253

254 To read the technical report: scan *Verdict* first (PASS / PASS-with-warnings / FAIL), then *Findings & warnings* ? each finding has a plain-English message, evidence, and a ? see README#Qn` pointer. To turn reports off or relocate them, set *report*: {"enabled": false} or *report*: {"output_dir": "..."} in *config.json*.

255

256 ### Q13: A validation check failed and indexing was halted ? what do I do?

257 Open the report's *Processing stages ? validation* block (or the *VALIDATION SUMMARY* printed to the console). Each failed check states the exact reason. Common cases: *relevance* (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); *resolution/fps* (below the configured minimums ? adjust *validation.min_* in *config.json*); *format* (not an MP4 container); *keyframes/PTS* (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in *config.json*, then re-run.

258

259 ---

260

261 ## ? Annotation & Evaluation

262

263 Once a video is processed, you can measure *how well* the pipeline detected its rallies against hand-annotated ground truth. You annotate each rally's [start, end) time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.

264

265 ```bash

266 python run_eval.py --scaffold --annotations rallies.csv # make an empty template

267 # ...fill in start,end rows, then:

268 python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-failures

269 ```

270

271 It reports precision/recall/F1, mean IoU, and boundary error for *two stages* ? raw detector *candidates* vs. AI-confirmed *final* segments ? which pinpoints whether a weak result

```

is a *detection* problem or a *segmentation* problem.
272
273     > Rally timestamps evaluate and tune **segmentation**; they do **not** train the shuttle
detector (that needs per-frame coordinate labels). See
**[docs/EVALUATION.md](docs/EVALUATION.md)** for the annotation format, CLI flags, and how to
read the output.
274
275     ---
276
277     ## ? Running the Test Suite
278     The test suite uses `pytest` and lives in `tests/`. From the project root:
279
280     ```bash
281     pip install pytest pytest-cov --user
282     pytest                                # run everything
283     pytest tests/test_yolo_hybrid.py      # run a single file
284     pytest -k yolo --maxfail=1 -v         # filter by keyword, fail fast, verbose
285     pytest --cov=backend --cov-report=term-missing # coverage report (uses .coveragerc)
286     ```
287
288     The suite mocks external dependencies (the Gemini client, ffmpeg/ffprobe, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
289
290     * `tests/test_base.py` ? segmenter registry
291     * `tests/test_database.py` ? SQLite schema, video/segment insert/query
292     * `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
293     * `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
294     * `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry
and the Gemini provider
295     * `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
296     * `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
297     * `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key
bailout, and timestamp-validation drops
298     * `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL
runner (mocked subprocess) and the `tracknet_hybrid` segmenter
299     * `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` +
`tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB
scoring, CLI)
300
301     ---
302
303     ## ? Project Structure (high level)
304
305     ```
306     backend/
307     main.py                # FastAPI app + CLI orchestrator
308     database.py            # SQLite schema and queries
309     validators/            # Pluggable video validators (format, blur, scene-cut,
relevance, ...)
310     indexer/              # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid
311     detectors/            # TrackNet WSL2 runner (shuttle-trajectory subprocess adapter)
312     providers/            # Pluggable AI providers (gemini today)
313     sports/              # Pluggable sport profiles (badminton today)
314     eval/                # Rally evaluation harness: metrics, annotations, scoring
315     stitcher/compiler.py   # FFmpeg-based highlight compiler
316     utils/               # ffprobe/ffmpeg + bbox geometry helpers
317     static/              # Zero-build frontend served by FastAPI
318     docs/                # DECISIONS (ADRs), TRACKNET_WSL_SETUP, EVALUATION
319     tests/               # pytest suite (mocked, runs offline)
320     run_process_and_stitch.py # Interactive end-to-end runner
321     run_eval.py          # Rally evaluation CLI (see "Annotation & Evaluation" above)
322     setup.py             # Dependency & FFmpeg diagnostics + default config init
323     config.json          # Runtime configuration (see "Threshold Tuning" above)
324     ```

```

4. user (2026-06-07T08:14:00.440Z)

```

1      # Badminton Highlight Indexer ? Video Report
2
3      ## Run
4
5      | Field | Value |
6      | --- | --- |
7      | Tool | badminton-highlight-indexer |
8      | Video id | md5b (file_md5) |
9      | Video path | /videos/match.mp4 |
10     | Generated | 2026-06-07T08:10:50Z |
11     | Schema | 1.0 |
12
13     ## Video
14
15     | Field | Value |
16     | --- | --- |
17     | Duration | 11m16s |
18     | Resolution | 1920x1080 |
19     | FPS | 59.94 fps (probed) |
20     | Container | mov,mp4,m4a |
21
22     ## Processing stages
23
24     **validation** ? `ok` (0.0s)
25     - checks_passed: 2
26     - checks_total: 2
27
28     **segmentation** ? `ok` (0.0s)
29     - candidate_windows: 0
30     - segment_count: 1
31     - total_play_s: 5 s
32     - _segmenter_requested_: motion
33     - _segmenter_actual_: motion
34     - _config_default_: motion
35     - _segmenter_explicitly_requested_: True
36     - _matches_config_default_: True
37     - _was_reingest_: False
38     - _detector_: motion
39
40     **ai_handoff** ? `skipped` (0.0s)
41     - _ai_based_: False
42     - _provider_: None
43     - _model_: None
44     - _api_key_present_: None
45
46     ## Findings & warnings
47
48     - [WARN] **synthetic_motion_metadata** ? The 'motion' segmenter was used. Its per-
segment metadata (intensity, speed, events, server/receiver) is HEURISTIC / synthetic, not
measured ? do not treat it as ground truth. Use yolo_hybrid/gemini for real shot metadata.
49     ? see **README#Q10**
50     ? evidence: stages.segmentation.details.segmenter_actual=motion
51
52     ## Verdict
53
54     **PASS (with warnings)**
55

```

5. assistant (2026-06-07T08:14:02.854Z)

Now I'll provide the structured output based on my analysis of the README and the technical

report.

6. user (2026-06-07T08:14:13.214Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

7. user (2026-06-07T08:14:21.189Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

8. assistant (2026-06-07T08:14:22.642Z)

Let me call it properly with all required fields:

9. user (2026-06-07T08:14:30.447Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

10. assistant (2026-06-07T08:14:31.793Z)

I need to provide the actual parameters. Let me call it correctly:

11. user (2026-06-07T08:14:40.582Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

12. assistant (2026-06-07T08:14:42.259Z)

I see the issue ? I need to actually pass the parameters. Let me do that now:

13. user (2026-06-07T08:14:47.829Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

14. assistant (2026-06-07T08:14:49.208Z)

Let me check the tool schema and properly format the call. I'll now construct the proper JSON structure with all required fields:

15. user (2026-06-07T08:14:58.781Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

16. assistant (2026-06-07T08:15:07.715Z)

I need to provide actual parameter values. Based on my analysis of the README and report, here is the diagnosis:

****Diagnosis:**** The run used the `motion` segmenter (a zero-dependency OpenCV pixel-difference heuristic), which successfully detected 1 segment totaling 5 seconds of play. However, the report includes a warning that the ****per-segment metadata is synthetic/heuristic, not measured****

? the `intensity`, `avg\_speed\_kmh`, serve/smash/foul/winner events, and `server`/`receiver` fields do not reflect ground truth. They are placeholder/generated values.

****Fix:**** To get real, measured metadata instead of synthetic heuristic values, switch from the `motion` segmenter to either:

- `yolo\_hybrid` (default) ? runs torchvision person detection + ByteTrack locally to filter candidates, then sends only the promising segments to an AI provider (Gemini) for precise boundaries and shot counts
- `gemini` ? sends the entire video directly to Google Gemini for analysis

Update `config.json` to set `"default\_segmenter": "yolo\_hybrid"` (or explicitly run with `--segmenter yolo\_hybrid`), and ensure you have a `GEMINI\_API\_KEY` in your `.env` file. Then re-run the indexing.

****FAQ section:**** Q10 ? "Can I trust the per-segment metadata from the `motion` segmenter?" This Q&A explicitly warns that metadata from the motion segmenter is heuristic/synthetic and explains that to get real values, you should use `yolo\_hybrid` or `gemini`.

****Could self-debug:**** Yes, I could self-debug this from the report. The warning message itself names the problem ("synthetic_motion_metadata") and explicitly cites README#Q10, which fully explains the issue and the solution.

****Confusing parts:****

- The report says validation passed and segmentation produced 1 segment, so it's not immediately obvious **why** this is a problem without reading the warning section carefully.
- The term "synthetic" could be clearer ? it might be easier to understand if the warning said "placeholder/unmeasured" upfront rather than burying "not measured" at the end of the sentence.
- The report shows `candidate\_windows: 0` but `segment\_count: 1`, which is slightly unintuitive ? it's unclear at first whether this is a success or a failure state without reading the full context about what the motion segmenter does.

Now calling StructuredOutput with these values:

17. user (2026-06-07T08:15:10.548Z)

Structured output provided successfully

18. assistant (2026-06-07T08:15:15.677Z)

Perfect. I've completed the diagnosis using ONLY the README and the technical report as instructed. Here's the summary:

What Went Wrong

The processing run used the `motion` segmenter, which is a zero-dependency OpenCV pixel-difference baseline. While it successfully detected 1 segment of 5 seconds of play, ****all per-segment metadata is synthetic/heuristic, not measured****. Fields like `intensity`, `avg\_speed\_kmh`, events (serve/smash/foul/winner), and `server`/`receiver` are placeholder values generated by a heuristic formula?they do not reflect ground truth.

Exact Fix

Switch to a segmenter that measures metadata:

1. Set `"default\_segmenter": "yolo\_hybrid"` in `config.json` (or run with `--segmenter yolo\_hybrid`)
2. Ensure `GEMINI\_API\_KEY` is in your `.env` file
3. Re-run: `python -m backend.main --video-path "C:/videos/match.mp4" --segmenter yolo\_hybrid`

The `yolo\_hybrid` segmenter runs torchvision person detection + ByteTrack locally to filter candidate windows, then sends only promising segments to Gemini for precise boundaries and real shot counts?avoiding full-video upload costs while getting measured metadata.

FAQ Used

****Q10: Can I trust the per-segment metadata from the `motion` segmenter?**** ? This Q&A explicitly states metadata is synthetic and directs you to use `yolo\_hybrid` or `gemini` for real measurements.

Self-Debug Assessment

****Yes****, I could fully self-debug this from the report alone. The warning explicitly cites README#Q10, which provides the complete explanation and fix.

Confusing Elements

1. ****Surface appearance of success**** ? The report shows "PASS (with warnings)" and 1 segment detected, so the problem isn't obvious without reading the warnings carefully.
2. ****Term "synthetic" is vague**** ? Words like "placeholder" or "unmeasured" would signal the issue faster than burying "not measured" at the end of a sentence.
3. ****Unintuitive metrics**** ? The report shows `candidate\_windows: 0` but `segment\_count: 1`, which is unclear without understanding the motion segmenter's internal behavior.